

InvoiceIQ — Final Consolidated PRD

Project Codename: Opaque × InvoiceIQ — Vulnerability-Hardened Edition v1.0

Lead Architect: M Darshan Kumar

Theme: AI for Fintech (Systemic Innovation & “Tech for Good”)

Build Window: 24-Hour Hackathon

Document Status: Final · Submission-Ready

§ 00 — Executive Summary

InvoiceIQ is a privacy-preserving AI early-warning system that predicts MSME cash flow collapse 30–60 days before it happens — and automatically generates the interventions to prevent it. ₹7.34 lakh crore is currently locked in delayed receivables across India’s 6.4 crore MSMEs. Every existing solution (TReDS, Samadhaan, invoice financing) activates *after* the crisis has begun, by which point the MSME has already taken informal credit at 36%+ interest. InvoiceIQ shifts the entire paradigm from reactive collection to predictive survival.

What separates this system from any other product in the market is its cryptographic architecture: **the data the AI analyzes is never seen in plaintext, by anyone, including us.** All buyer-risk inference runs on ciphertext via Fully Homomorphic Encryption. Aggregate network signals are protected by Differential Privacy. The informal cash economy — the reality of how MSMEs actually operate — is bridged through a Telegram NLP layer with Maker-Checker controls.

This PRD is the consolidated, vulnerability-hardened final version of all prior plans. Every feature has been preserved, merged, or upgraded with a documented replacement. Every cryptographic claim has been scoped honestly. Every legal citation has been verified. Every demo failure mode has been pre-mitigated.

§ 01 — The Problem: Six Structural Failures

The MSME cash flow crisis is not a single problem. It is six interlocking failures that no single existing tool addresses:

Failure 01 — Reactivity. TReDS, Samadhaan, and invoice financing only activate after an invoice is already overdue. By then the MSME has taken emergency credit at 36% APR. The damage is structural.

Failure 02 — The Retaliation Trap. Section 15 of the MSMED Act gives MSMEs the right to demand timely payment. Section 43B(h) of the Income Tax Act penalizes buyers who don't pay on time. But documented evidence shows large buyers cancel orders or move business to unregistered suppliers when MSMEs assert these rights. The law gives the right; the market punishes its use.

Failure 03 — Data Surrender. Every fintech analytics platform requires uploading complete books to a third-party server. For an MSME, exposing buyer relationships, pricing, and revenue concentration to a breachable cloud is existentially risky.

Failure 04 — Hidden Contractual Traps. Buyer agreements contain dispute-trigger clauses, set-off rights, force-majeure loops, and invoicing-defect resets that legalize indefinite payment delays. MSMEs sign without legal review.

Failure 05 — Decryption-in-Memory. Even encrypted-at-rest data must decrypt to RAM for AI analysis. That moment is the attack surface for RAM scrapers, AI malware, and insider threats. FHE eliminates this attack class entirely.

Failure 06 — Informal Economy Reality. Indian MSMEs operate in a mixed cash-digital world. Petty cash, vendor payments, employee advances — none of it touches a digital ledger. Any survival prediction built only on bank data is missing 30–50% of actual cash movement.

§ 02 — Goals & Success Metrics

The system targets five measurable outcomes for the hackathon demo and beyond:

Predict cash flow collapse 30–60 days in advance with 80% confidence intervals on the runway figure. Process Telegram cash logs from informal employees in under 3 seconds end-to-end. Run buyer reliability inference entirely on FHE-encrypted numeric features with zero plaintext server

exposure. Detect the four defined contract dark-pattern clause types with 100% recall on the curated demo contract. Generate legally accurate intervention notices that correctly cite Section 15 of the MSMED Act 2006 and Section 43B(h) of the Income Tax Act 1961 with zero LLM-introduced hallucinations.

§ 03 — Target Users

Primary: Indian MSME owners running businesses with 2–50 employees, operating in mixed digital/cash workflows, lacking deep financial literacy, and facing systemic retaliation when asserting legal rights against larger corporate buyers.

Secondary: MSME employees who handle petty cash and vendor payments on the owner’s behalf, and need a frictionless way to log expenses through the channel they already use (Telegram/WhatsApp).

The Reality Addressed: Users do not have clean digital ledgers. They cannot afford a CA on retainer. They will not upload their books to a cloud they don’t trust. The product must work *with* the informal economy, not against it.

§ 04 — Core Architecture: Six Integrated Layers

Layer 1 — Hybrid Cash-Digital Ledger

The Telegram Bot acts as the primary input channel for informal cash movements. An employee texts “*Paid 5k cash to Raju for transport*” and a local Llama 3 instance (running through Ollama in JSON-output mode) extracts a structured intent: `{action: "expense", amount: 5000, vendor: "Raju", category: "transport"}`. A regex fallback parser catches any case where Llama returns malformed JSON, ensuring the bot never silently fails. A vendor alias table maps colloquial names to canonical vendor IDs to prevent duplicate entries.

Every transaction over ₹10,000 is automatically held in a separate `pending_transactions` table and pings the MSME owner for one-tap approval before being committed to the main ledger. This Maker-Checker control prevents both employee error and deliberate sabotage. Pending transactions are explicitly excluded from runway math and clearly displayed on the dashboard as

“awaiting approval.” End-of-day cash reconciliation lets the owner enter till totals, and the system computes unrecorded cash sales by reverse-balancing the day’s known movements.

The ₹10,000 threshold is the sensible default for the demo and is configurable post-launch per MSME segment.

Layer 2 — Predictive Buyer Reliability Engine

For each buyer, the system computes payment behavior using methods that respect the actual data distribution. **Exponential Moving Average and Payment Velocity** are used instead of simple means, so the model snaps quickly to recent liquidity changes. **Variance is captured via the empirical 90th percentile** of historical delays, not $\mu + 1.5\sigma$ — payment delay distributions are heavily right-skewed (lognormal), and Gaussian assumptions systematically underestimate worst cases. **The Cold-Start Protocol** maps unknown buyers to a sector-baseline lookup table (textile, auto-ancillary, FMCG, etc.) derived from RBI MSME Pulse data, replacing arbitrary risk haircuts with defensible numbers.

Meta’s Prophet is used **only for aggregate cash inflow forecasting** (where the time series has enough data points to be reliable), not for per-buyer prediction. Per-buyer trend math uses EMA and velocity, which are appropriate for the 6–12 invoice samples that real MSMEs have per buyer per year.

The ML core is an **XGBoost Buyer Reliability classifier** compiled to FHE-compatible form via Zama’s concrete-ml. Tree depth is capped at 4 and inputs are quantized to 8 bits to keep inference latency in the 5–30 second range per query, which is then handled asynchronously.

Layer 3 — Cash Flow Collapse Predictor (Survival Runway)

The runway is a deterministic calculation, kept in plaintext on the client side because FHE adds zero security value to local arithmetic on data the user already owns:

Risk-Adjusted Receivables = $\Sigma(\text{invoice_amount} \times (1 - \text{overdue_probability}))$ for all open invoices.

Survival Runway = $(\text{Digital Reserves} + \text{Cash Reserves} + \text{Risk-Adjusted Receivables}) / \text{Daily Burn Rate}$.

Critically, the runway is never displayed as a single number. It always carries an 80% confidence interval derived from Monte Carlo sampling over the buyer probability distributions. The dashboard shows: **“34 Days · 80% CI: 22–48 Days.”** This single design choice converts the product from “AI gimmick” to “scientifically defensible tool” and pre-empts the inevitable judge question about uncertainty.

A **Concentration Risk** metric runs alongside the runway: *“67% of your runway depends on Apex Manufacturing.”* This is a one-line Pandas calculation that surfaces the most actionable insight any MSME owner can have.

Layer 4 — Contract Dark Pattern Detector

When the user uploads a buyer agreement PDF, pdfplumber extracts the text and a **spaCy pattern matcher with an expert-curated rule library** identifies the four critical clause types: dispute-triggered payment holds, set-off rights, force-majeure payment loops, and invoicing-defect nullification clauses. This is rule-based by deliberate choice — fine-tuning BERT in 24 hours is infeasible, dataset-free, and unreliable, while pattern matching is fast, has 100% recall on the curated demo contract, and is exactly how real legal-tech tools operate. Each detected clause is shown with severity, plain-English explanation, and rupee impact estimate over the contract tenure.

Layer 5 — Hybrid Retaliation Detector

This is the layer that exists nowhere else in the market. The system tracks order volume per buyer in a 90-day rolling window. When a legal action is taken (notice sent, complaint filed), the detector compares pre-event and post-event order volume using a **deterministic two-sigma test**: if the 30-day post-event mean drops more than 2σ below the 90-day pre-event mean, the system flags retaliation with an explicit p-value. An **Isolation Forest** layer is added on top *only* for buyers with more than 50 historical invoices — sparse-data anomaly detection produces too many false positives to be trustworthy below that threshold.

When triggered, the system documents the pattern, generates a pre-filled MSME Ministry complaint, and suggests buyer diversification based on the MSME’s industry cluster.

Layer 6 — Project Opaque (Privacy & Security Core)

The cryptographic foundation has three precisely-scoped components:

Fully Homomorphic Encryption (Zama concrete-ml) is applied strictly to the Buyer Reliability scoring model. Numeric features (amounts, days, deltas, ratios) are encrypted client-side and the XGBoost model runs inference on ciphertext server-side. The architectural purpose is to enable **future federated cloud aggregation** without exposing individual MSME books — *not* to provide local-only security, where FHE would add no benefit. This framing is honest, defensible, and matches the technology’s actual capability.

AES-GCM symmetric encryption is applied to all string fields (buyer names, GSTINs, invoice references, vendor names) that FHE cannot natively handle. This combination — FHE for numeric ML inputs, AES-GCM for metadata — is the standard production pattern and must be articulated explicitly in the demo.

Differential Privacy replaces the previously-considered ZK-SNARK layer, which is research-grade and not implementable in 24 hours. When an MSME opts into the federated buyer reputation network, Laplace noise ($\epsilon = 1.0$, calibrated to invoice-amount sensitivity) is added to defaulted invoice values before aggregation. This preserves the statistical signal across thousands of submissions while making any individual transaction non-recoverable.

Encrypted USB Backup provides offline data sovereignty: a one-click export of the SQLite state as an AES-256 encrypted archive. This is a five-minute build that produces an outsized trust signal during demo Q&A.

Asynchronous FHE Caching addresses the 5–30 second inference latency: a Celery background worker pre-computes FHE inferences and stores ciphertext results in a cache table. The Streamlit UI polls only the cache, ensuring zero demo lag. When a new invoice is logged, the stale cache is flagged and recomputed in background while the UI shows “Risk score updating...” rather than freezing.

SHAP explainability runs client-side on the decrypted feature subset and a stored model-feature-importance lookup, producing per-buyer waterfall charts that explain why a buyer’s reliability score is what it is. This is critical for converting AI output from black-box to defensible.

§ 05 — User-Facing Features (Tiered for 24-Hour Build)

Tier 1 — Must Ship (Demo-Critical)

The Streamlit Command Center launches in dark-theme wide mode with a massive animated **Survival Runway Gauge** showing days-to-collapse with 80% confidence interval. The **What-If Slider** lets the user simulate “if Buyer X delays by N days” and watch the runway recompute live. **Buyer Reliability Cards** present the 6–8 buyers with score, trend arrow, and a click-through to a pre-computed SHAP waterfall. **Telegram Cash Logging** with the Maker-Checker pending-approval workflow. **Auto-Generated Legal Notice** as a downloadable PDF citing Section 15 MSMED Act 2006 and Section 43B(h) IT Act 1961, with all legally-substantive content from Python f-string templates and only the cover paragraph tone-shaped by Llama 3. **The FHE Architecture Panel** displays ciphertext-in / ciphertext-out / inference latency in a structured format — proving real computation, not a hex wall. **Concentration Risk Metric** as a top-level dashboard tile.

Tier 2 — Ship if Ahead of Schedule

Contract Dark Pattern Detector with the four clause types. Hybrid Retaliation Detector with the two-sigma test and pre-filled ministry complaint. Encrypted USB Backup one-click export. Differential Privacy demonstration with before/after Laplace-noise visualization on a synthetic federated submission.

Tier 3 — Ship Only if Flying

PyVis Contagion Graph capped at 20 nodes, rendered as a **pre-generated static HTML** triggered by button click — never as a live `time.sleep/st.rerun` animation, which is fragile and freezes Streamlit. Whisper-based voice input through Telegram for owners with low literacy. Full audit-trail PDF report. TReDS eligibility auto-checker. Samadhaan complaint pre-fill.

The Tier Discipline Rule

Tier 1 stability is never sacrificed for Tier 2 features. Tier 2 is never sacrificed for Tier 3. If at Hour 17 Tier 1 isn't fully stable, Tier 2 work stops immediately and the team converges on demo polish.

§ 06 — Technology Stack

Domain	Technology	Purpose
Frontend	Streamlit (wide/dark), Plotly, PyVis	Command Center, animated runway, optional contagion graph
Backend	Python, SQLite (local), FastAPI, Celery	Local-first state, async FHE orchestration
ML/Data	Pandas, scikit-learn, XGBoost, Prophet, SHAP	Buyer reliability, aggregate forecasting, explainability
NLP	Ollama + Llama 3 (JSON mode, pinned version), spaCy, pdfplumber, RapidFuzz	Cash log parsing, contract clause detection, vendor alias matching
Crypto	Zama concrete-ml (FHE), PyCryptodome (AES-GCM), custom Laplace-noise module	Encrypted ML, metadata encryption, differential privacy
Integration	Telegram Bot API (primary), Twilio WhatsApp (production roadmap)	Informal cash logging
Synthetic Data	Faker (hi_IN locale) + curated Indian MSME name list	Demo realism with fixed seed

Every component is open-source and free. No paid APIs. No external dependencies that judges can't reproduce.

§ 07 — 24-Hour Execution Plan

Hours 0–3 — Foundation. Install full stack (Concrete-ML compilation tested first, since this is the highest-risk component). Generate 5,000 synthetic invoices across 6–8 buyers with Faker hi_IN locale and a fixed random seed: one consistent 45-day payer, two erratic high-variance payers, one cold-start unknown, and one pre-scripted retaliation case with a hard-coded 83% volume drop after a specific date. Stand up SQLite schemas for buyers_registry, invoices_ledger, cash_transactions, pending_transactions, and fhe_cache. Pull Llama 3 via Ollama at a pinned version (llama3:8b-instruct-q4_0). Configure Telegram Bot webhook.

Hours 3–9 — Core ML & Math. Train the XGBoost Buyer Reliability model on synthetic data and compile to FHE via concrete-ml. Build the EMA + payment velocity per-buyer module. Implement the empirical 90th-percentile variance calculation. Wire the sector-baseline lookup table. Build the deterministic Survival Runway calculator with Monte Carlo confidence intervals. Implement the Concentration Risk metric. Test end-to-end: invoice in → encrypt → FHE inference → decrypt → display with SHAP.

Hours 9–14 — Cash Layer & Interventions. Wire the Telegram bot with Llama 3 JSON-mode parsing and regex fallback. Build the pending-transactions Maker-Checker workflow with explicit exclusion from runway math. Implement EOD reconciliation. Build f-string-based legal notice templates with verified Section 15 MSMED + Section 43B(h) IT Act citations, and the constrained Llama 3 tone-paragraph generator. Build the deterministic two-sigma retaliation detector.

Hours 14–18 — Privacy Stack & UI. Wrap the model in concrete-ml with the asyncio Celery cache pre-computer. Implement AES-GCM string encryption. Build the Differential Privacy Laplace-noise module for federated submission demo. Build the FHE Architecture Panel showing ciphertext flow. Implement the Encrypted USB backup. Build the What-If slider. Wire SHAP waterfalls (pre-computed for demo buyers). Build Tier 2 contract dark pattern detector with spaCy if time permits.

Hours 18–21 — Sad-Path Testing & Hardening. Run the full demo end-to-end three times back-to-back. Trigger every error path: malformed Telegram input, missing buyer, FHE cache miss, oversized PDF upload, Maker-Checker rejection, empty database state. Capture fallback screenshots for every Tier 1 step. Verify every legal citation against the Acts. Pre-compute and cache all SHAP timelines for demo buyers. Hard-code the synthetic-data narrative so the retaliation alert fires at exactly the demo moment.

Hours 21–24 — Demo Polish & Submission. Rehearse the 5-minute pitch three times, timing each section. Write the README with explicit FHE-scope disclosure, ZK-deferral note, and tier description. Build the 5-slide deck. Submit.

§ 08 — 5-Minute Demo Script

Minute 1 — The Hook. Open with the Supabase/SQLite ciphertext view. “₹7.34 lakh crore is locked out of India’s small businesses. Rohan has 12 employees and delivered on time 67 days ago. He hasn’t been paid. Every existing tool requires him to upload his entire financial life to a stranger’s server. We built a system that sees his future without ever seeing his data. Even if our database is fully exfiltrated, an attacker holds only ciphertext — and FHE eliminates the decryption-in-memory attack class entirely.” (Note: softened from the original “hack us” framing — judges with security backgrounds will respect this calibration.)

Minute 2 — The FHE Reveal. Run the encryption script live. Show plaintext on the left, ciphertext generation on the right, the FHE Architecture Panel showing inference happening on encrypted features, and the result returning. Emphasize: numeric features via FHE, strings via AES-GCM, all client-decryptable.

Minute 3 — The Runway and What-If. Switch to the Streamlit dashboard. Show “47 Days · 80% CI: 32–58” with the concentration risk tile. Click Apex Manufacturing’s buyer card → show the SHAP waterfall explaining the score drop from 81 to 53. Drag the What-If slider: “*What if Apex pays 20 days late?*” Watch the runway collapse to 19 days in real time.

Minute 4 — Intervention and Dark Patterns. Click Generate Intervention Bundle. Show the legal notice PDF with the correct Section 15 MSMED Act and Section 43B(h) IT Act citations. Upload the synthetic Apex contract. Watch spaCy flag the dispute-trigger clause and the invoicing-defect reset clause in 8 seconds, with rupee-impact estimates.

Minute 5 — Retaliation, Cash Layer, Close. Show the retaliation alert: order volume from Apex dropped 83% post-notice, $p < 0.02$. Show a Telegram message “Paid 5k to Raju” being parsed and logged in real time, with a ₹15,000 transaction triggering the Maker-Checker pending state. Close on the northstar: “*InvoiceIQ doesn’t sell financing. It sells certainty — manufactured from data we cannot read. The law gives MSMEs the right to be paid. We give them the foresight to use that right safely. Rohan makes payroll.*”

§ 09 — Risk Register

Risk	Severity Probability		Mitigation
FHE inference too slow for live demo	High	Medium	Pre-compute all FHE results during setup; UI displays cached ciphertext outputs. Architecture is live; computation is async. Disclose openly.
Concrete-ML compilation fails on demo laptop	High	Low	Compile artifact committed to repo. Test on demo hardware 12 hours before. Fallback: show pre-recorded compilation output.
Llama 3 returns malformed JSON	Medium	Medium	JSON output mode + regex fallback parser. Pinned model version. Vendor alias table prevents duplicate entries.
Legal citation error caught by judge	High	Low	Manual audit of every Section 15 / Section 43B(h) reference before submission. F-string templates only — no LLM-generated statute references.
Streamlit auto-refresh stalls	Medium	Medium	st.cache_data for SQLite reads, Celery for heavy compute, 3–5 sec poll interval (not 2).

Risk	Severity Probability		Mitigation
Telegram bot fails during demo	Medium	Low	Pre-recorded message replay script as fallback. Bot tested on demo network 1 hour before.
Sparse-data false positives in retaliation detector	Medium	Low	Isolation Forest gated to buyers with >50 invoices; deterministic 2σ test for everyone else.
Judge questions ZK-SNARK absence	Low	Medium	Explicit, confident framing: “ZK-SNARKs are research-grade and not 24-hour buildable. Differential Privacy achieves the federated-aggregation privacy goal with deployable mathematics. We chose honest engineering over crypto theater.”
Demo synthetic data looks fake	Medium	Medium	Faker hi_IN + curated Indian MSME name list + GSTIN format validation + fixed seed. Hard-coded retaliation storyline.
FHE+Streamlit memory crash	Medium	Low	Celery workers isolate FHE memory. UI process stays light. Tested with 6–8 concurrent buyer scores.

§ 10 — Vulnerability Audit: Before vs After

The consolidation process explicitly resolved every vulnerability identified across prior plans. The decryption-in-memory attack class is eliminated by FHE on numeric features paired with AES-GCM on metadata. The previously over-claimed ZK-SNARK layer is replaced with deployable Differential Privacy. The arbitrary 70% cold-start risk haircut is replaced with sector-baseline lookups grounded in RBI data. The Gaussian variance assumption ($\mu + 1.5\sigma$) is replaced with the empirical 90th percentile, which respects the actual lognormal distribution of payment delays. The sparse-data Isolation Forest false-positive risk is gated by the >50-invoice threshold. The Maker-Checker race condition is closed by the separate pending_transactions table excluded from runway math. The fragile time.sleep/st.rerun contagion animation is demoted to Tier 3 and reimplemented as static pre-rendered HTML. The “Live FHE Terminal” hex-wall theater is replaced with a structured panel showing inference latency. The legal-text hallucination risk is eliminated by f-string templating with Llama 3 constrained to tone-paragraph only. The Section 43B(h) / Section 15 conflation is corrected throughout: 43B(h) is the Income Tax Act 1961 deduction-disallowance rule; Section 15 is the MSMED Act 2006 45-day rule; both are cited in the correct contexts. The “Hack us right now” demo framing is softened to a calibrated, defensible statement that doesn’t invite questions about endpoint, session, or browser-localStorage attack vectors that FHE doesn’t address. The fine-tuned BERT contract classifier is replaced with spaCy pattern matching, which is faster, dataset-free, and gives 100% recall on the curated demo contract. The Twilio WhatsApp

dependency is replaced with Telegram (no business-account approval friction), with WhatsApp listed as a production-roadmap item.

§ 11 — Post-Hackathon Roadmap

Months 1–3. Integrate with the Account Aggregator framework for real bank-statement ingestion, replacing manual CSV upload. Integrate the GST API for live invoice and buyer GSTIN validation. Optimize FHE inference latency using Intel HEXL acceleration. Expand the dark-pattern taxonomy from four to twenty clause types based on real MSME contract corpus.

Months 3–6. Pilot deployment with two MSME industry associations (textile cluster, auto-ancillary cluster). Pursue SIDBI / GAME partnership for credibility and validation data. Launch B2B SaaS API: white-label the Buyer Reliability Score for NBFCs to use in their MSME lending underwriting. Direct revenue model at ₹499/month per MSME; B2B at ₹2–5 per score API call.

Year 1+. Open-source the FHE financial-data layer as public infrastructure, establishing a privacy-preserving financial data standard for India. Pursue TReDS API integration for one-click discounting. Publish the aggregated retaliation-intelligence dataset (anonymized via Differential Privacy) as the first empirical map of corporate retaliation against MSMEs — converting it from a private tool into a public-accountability instrument.

§ 12 — Why This Submission Wins

The cryptographic depth is genuine: FHE + AES-GCM + Differential Privacy is a coherent, defensible, deployable privacy stack — not crypto theater. Five independent intelligence layers (predictive ML, FHE privacy, contract NLP, retaliation detection, informal-cash bridge) tell one integrated story that no single-feature submission can match. The demo emotion — Rohan making payroll, the runway collapsing live, the retaliation alert firing — produces beats that anomaly-detection dashboards cannot replicate. Every claim in this PRD is implementable in 24 hours by a 3–4 person team with the discipline to follow the Tier system. Every legal citation is verified. Every cryptographic claim is correctly scoped. Every demo failure mode has a documented mitigation.

The northstar: *The MSME cash flow crisis is not a capital problem — it is a certainty problem. Businesses can survive a 60-day payment delay if they know it's coming. They cannot survive a 5-day delay they didn't see. InvoiceIQ sells certainty, manufactured from encrypted data that no one — not even us — can read.*

Document Status: Final · Vulnerability-Hardened · Submission-Ready

Approved Build Stack: 100% Open Source · Zero Paid APIs · 24-Hour Buildable

Last Audit: All Section 15 MSMED Act 2006 and Section 43B(h) Income Tax Act 1961 citations verified · All cryptographic scope claims honestly bounded · All demo failure modes documented with mitigations